

Dimension Reduction

John Snyder

STAT 9340: Spring 2023

Section 1

Autoencoders

Autoencoders

An autoencoder is an **unsupervised** neural network that attempts to find functions that transform inputs to generate a reconstruction of the input

- The output is the input!

Originally, these models were used to find interesting relationships and dimension reduction and information retrieval in the inputs, which is how we will look at them here for the most part.

But, as we will see later in the class, they have taken on broader utility in recent years.

Autoencoders

Imagine one has an input vector $\mathbf{x}$. This is transformed (encoded) through a function $f(\cdot)$ to obtain a “hidden layer:”

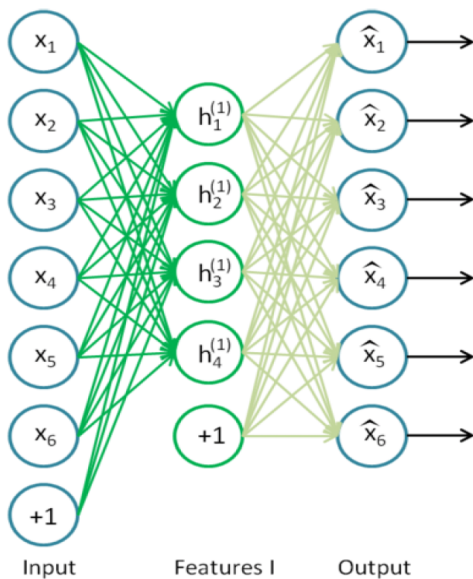
$$\mathbf{h} = f(\mathbf{x}),$$

the dimension of which is much lower than the dimension of $\mathbf{x}$. This hidden layer is then reconstructed by a (decoder) function $g(\cdot)$,

$$\hat{\mathbf{x}} = g(\mathbf{h})$$

where $\hat{\mathbf{x}}$ is close to $\mathbf{x}$: i.e., $g(f(\mathbf{x})) \approx \mathbf{x}$. The model is forced to prioritize the aspects of the input to be copied and so, in principle, learns useful properties of the data.

Autoencoder pic



Autoencoders (cont.)

Typically, the learning process for autoencoders consists of minimizing some loss function, e.g.,

$$L(\mathbf{x}, g(f(\mathbf{x}))),$$

where the loss function L penalizes the encoder for being dissimilar from $\mathbf{x}$ (examples include the negative log-likelihood; e.g., squared error for Gaussian, cross entropy for Bernoulli, etc.).

One way to obtain useful features is to constrain $\mathbf{h}$ to have a dimension less than $\mathbf{x}$; such an encoder is said to be *undercomplete*. When the decoder is linear and L is the MSE, an undercomplete encoder learns to span the same subspace as PCA. Of course, this isn't very interesting as we could just use PCA. But, it does let us think about the obvious nonlinear generalizations. In particular, autoencoders with nonlinear encoder and decoder functions, f and g , respectively, can learn more powerful dimension reduction and feature spaces.

Regularized Autoencoders

Autoencoders can also consider $\mathbf{h}$ of the same dimension as $\mathbf{x}$ (complete) or even $\mathbf{h}$ with a higher dimension than $\mathbf{x}$ (over-complete). In those cases, regularization or other constraints must be used or the autoencoder just learns the identity function. An obvious solution to the over-complete problem is the use of a regularization term in the optimization function or add some stochastic noise to the original inputs.

Tensorflow Code for a Basic Autoencoder

```
encoding_dim = 32 # << x_train.shape[1]

AEinput = tf.keras.Input(shape=(x_train.shape[1],))
hiddenLayer = layers.Dense(encoding_dim, activation='relu')(AEinput)
AEoutput = layers.Dense(x_train.shape[1], activation='relu')(hiddenLayer)

# Compile as usual
autoencoder.compile(
    optimizer=tf.keras.optimizers.Adam(),
    loss=tf.keras.losses.MeanSquaredError()
)

# training data passed as input and output!
history = autoencoder.fit(
    x_train, x_train,
    ...
)

autoencoder = Model(AEinput, AEoutput)
```

Regularized Autoencoders

Sparse Autoencoders: A natural form of regularization is achieved by adding a sparsity penalty to the loss function. E.g.,

$$L(\mathbf{x}, g(f(\mathbf{x}))) + \Omega(\mathbf{h}),$$

where $\Omega(\mathbf{h})$ is the sparsity penalty function. Such sparse autoencoders are typically used to learn features for another task, such as classification. Note, since $\mathbf{h}$ is a function of the input observations, one can't really think of the sparsity penalty as a “prior” as we have done in lasso and ridge regression. Yet, this still acts as a constraint on the possible solutions. However, note that one can think of the joint probabilistic model $p(\mathbf{x}, \mathbf{h}) = p(\mathbf{x} | \mathbf{h})p(\mathbf{h})$ and, in this case, $p(\mathbf{h})$ is essentially a prior on the latent process that influences the data. One could then choose something like a Laplace distribution for $p(\mathbf{h})$ to induce sparseness.

This can be integrated into a TF layer with e.g.

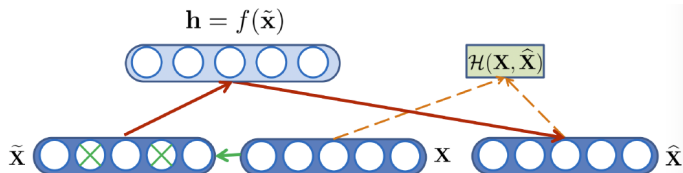
- `activity_regularizer=regularizers.l1(0.001)`

Regularized Autoencoders

Denoising Autoencoder (DAE): Instead of adding a penalty to the loss function, we can prevent the identity reconstruction problem by using a corrupted form of the input,

$$L(\mathbf{x}, g(f(\tilde{\mathbf{x}}))),$$

where $\tilde{\mathbf{x}}$ is a copy of $\mathbf{x}$ that has been “corrupted” by noise. In some cases, one can corrupt the input by setting many values to 0, in which case the autoencoder is attempting to learn how to predict the missing observations.



- Standard dropout can be used for this in TF
- We can also use the `tf.keras.layers.GaussianNoise()` layer

These layers are regularizers and are therefore only active during training.

Regularized Autoencoders

Contractive Autoencoder (CAE): Another approach for regularizing the autoencoder is use a penalty that is based on the gradient of the hidden units:

$$L(\mathbf{x}, g(f(\mathbf{x}))) + \Omega(\mathbf{h}, \mathbf{x})$$

where

$$\Omega(\mathbf{h}, \mathbf{x}) = \lambda \sum_i \|\nabla_{\mathbf{x}} h_i\|^2.$$

This penalty function pushes the model to learn a function that does not change much when $\mathbf{x}$ changes slightly. Note that there is a connection between the CAE and the DAE (see Alain and Bengio, 2013). Basically, DAEs make the reconstruction function resist small (finite-sized) perturbations to the input, where CAEs make the feature extraction function resist infinitesimal perturbations to the input.

Example

Consider a case where we have an p dimensional input vector, $\mathbf{x}$ and r dimensional hidden vector $\mathbf{h}$. Assume a sigmoid activation function for $f(\cdot)$, call it $\sigma(\cdot)$:

$$\mathbf{h} = f(\mathbf{x}) = \sigma(\mathbf{W}_e \mathbf{x} + \mathbf{b})$$

where $\mathbf{W}_e$ is the $r \times p$ encoding parameter matrix, and $\mathbf{b}$ is the r -vector of hidden layer biases. If the input features take binary values, we could use

$$\hat{\mathbf{x}} = g(\mathbf{h}) = \sigma(\mathbf{W}_d \mathbf{h} + \mathbf{a}),$$

where $\mathbf{W}_d$ is the $p \times r$ decoding parameter matrix, and $\mathbf{a}$ is the p vector of decoding layer biases. Note, if the input features take real values, then one can use

$$\hat{\mathbf{x}} = g(\mathbf{h}) = \mathbf{W}_d \mathbf{h} + \mathbf{a}.$$

So, $\mathbf{W}_e$, $\mathbf{W}_d$, $\mathbf{b}$, and $\mathbf{a}$ are parameters (weights) to be estimated. Note, in the context of dimension reduction, $\mathbf{h}$ is the dimension reduced feature set.

Regularization

As mentioned, for traditional autoencoder implementations, we often add a regularization term to shrink the weight parameters towards zero (e.g., an L_2 or L_1 penalty) and an overall “sparsity regularization” that shrinks the hidden units. For example, let

$$\hat{\rho}_j = \frac{1}{n} \sum_{i=1}^n g(\mathbf{w}'_j \mathbf{x}_i + b_j) = \frac{1}{n} \sum_{i=1}^n h_j(i),$$

be the average of the j th hidden unit averaged over the n samples in the training set (note, $\mathbf{w}'_j$ is the j th row of $\mathbf{W}_e$). One seeks to enforce the constraint that

$$\hat{\rho}_j \approx \rho$$

where ρ is a sparsity parameter, which is typically a small value close to zero (e.g., $\rho = 0.05$). So, we would like the average “activation” (value) of each hidden unit to be close to zero.

Regularization

One way to do this is to add an extra penalty, such as

$$\sum_j \rho \log \frac{\rho}{\hat{\rho}_j} + (1 - \rho) \log \frac{1 - \rho}{1 - \hat{\rho}_j} = \sum_j KL(\rho \| \hat{\rho}_j),$$

which is just the Kullback-Leibler (KL) divergence between a Bernoulli random variable with mean ρ and a Bernoulli random variable with mean $\hat{\rho}_j$. Recall, the Kullback-Leibler divergence is a measure of the difference between two distributions. In this case, it takes the value zero when ρ and $\hat{\rho}_j$ are equal to each other, and becomes larger as they diverge from each other. Minimizing the cost function forces this term to be small, hence ρ and $\hat{\rho}_j$ to be close to each other.

Thus, we can minimize an objective function that includes the loss as well as regularization on the weights and the sparsity,

$$Q(\mathbf{W}) = L(\mathbf{x}, g(f(\mathbf{x})); \mathbf{W}) + \lambda \Omega(\mathbf{W}) + \beta \Omega(KL(\rho \| \hat{\rho}_j))$$

Extreme Learning Machine (ELM) Autoencoders

Here, we consider an alternative (very simple) training approach for autoencoders based on a type of “reservoir computing” known as extreme learning machines (e.g., see the review in Huang et al. 2015). Assume we have a feature matrix $\mathbf{X}$ with elements in $\mathbb{R}^{p \times n}$. Now, we seek a lower-dimension representation of $\mathbf{X}$ via a matrix of hidden units $\mathbf{H} \in \mathbb{R}^{r \times n}$, where $r < p$, and we seek the matrix $\mathbf{W} \in \mathbb{R}^{p \times r}$ such that $\mathbf{X} \approx \mathbf{WH}$. The first step is to find $\mathbf{H}$ via:

$$\mathbf{H} = g(\mathbf{W}^0 \mathbf{X} + \mathbf{B}^0),$$

where $g(\cdot)$ is an activation function (e.g., a sigmoid, tanh, Gaussian, etc.), and importantly, the weights $\mathbf{W}^0 (r \times p)$ and $r \times n$ matrix, $\mathbf{B} = [\mathbf{b}^0, \dots, \mathbf{b}^0]$ (where the $r \times 1$ bias vector $\mathbf{b}^0$ is repeated n times) are selected randomly!

Extreme Learning Machine (ELM) Autoencoders

Next, we find the $\mathbf{W}^*$ from the optimization:

$$\mathbf{W}^* = \arg \max_{\mathbf{W}} \|\mathbf{WH} - \mathbf{X}\|_2 + \lambda \|\mathbf{W}\|_2.$$

So, this is just a multivariate ridge regression estimation problem given $\mathbf{H}$ from the previous step (which plays the role of X in our usual regression formulation), and where $\mathbf{X}$ plays the role of Y . So,

$$\mathbf{W}^* = \mathbf{XH}' (\mathbf{HH}' + \lambda \mathbf{I})^{-1}$$

Now, the final reduced data matrix, say $\mathbf{H}^*$ is represented by:

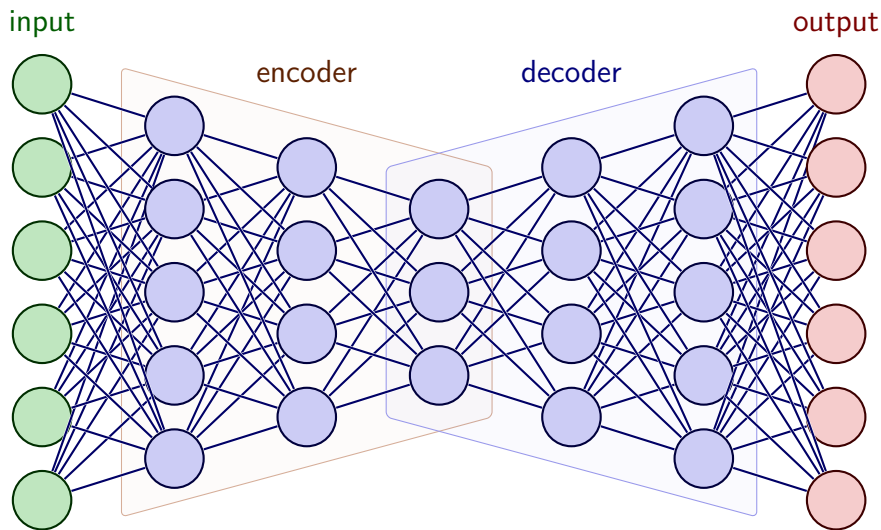
$$\mathbf{H}^* = (\mathbf{W}^*)' \mathbf{X}$$

and the reconstruction is given by:

$$\hat{\mathbf{X}} = \mathbf{W}^* \mathbf{H}^*$$

Stacked Autoencoders

Stacking a mirrored architecture allows learning more complex relationships.



Application: Anomaly Detection

Autoencoders can be used to detect anomalies in input data sources.

- If the model has never seen an input resembling a new measurement, it will not be able to reconstruct it. Take the below example, when some images are fed through an autoencoder trained on the MNIST dataset



The MSEs for these images are

## [1]	0.115	0.066	0.005	15207.059	0.037
--------	-------	-------	-------	-----------	-------